Machine Learning - Day 2

Linear Regression Deep Dive

Today's learning completes the Linear Regression foundation: cost function, Gradient Descent, learning rate, Normal Equation, train/test split, evaluation metrics, overfitting, underfitting, and regularization.

1. What Are We Actually Trying to Find?

Given training data, we assume an approximately linear relationship:

y_hat = b0 + b1*x

The algorithm must learn the best values of b0 and b1. Therefore Linear Regression can be viewed as an optimization problem: find parameters that minimize prediction error.

```
Training Data -> Find b0,b1 -> Model -> Predictions -> Error -> Cost -> Minimize Cost
```

2. Cost Function

A common cost function is:

J(b0,b1) = (1/(2n)) * SUM((y_i - y_hat_i)^2)

The factor 1/2 mainly makes differentiation cleaner. Since y_hat_i = b0 + b1*x_i, the cost depends on b0 and b1. Our objective is to minimize J(b0,b1).

3. Why Does Cost Depend on the Parameters?

Changing b0 changes the intercept. Changing b1 changes the slope. Therefore both parameters change predictions, which changes errors and finally the cost.

```
b0, b1 -> Predictions -> Errors -> Cost J(b0,b1)
```

4. Gradient Descent

Gradient Descent is an optimization algorithm that repeatedly adjusts model parameters in a direction that reduces the cost.

```
Initial Parameters
      |
Make Predictions
      |
Calculate Cost
      |
Calculate Gradient
      |
Update Parameters
      |
Repeat
```

Intuition: imagine standing on a hill and wanting to reach the lowest point. You determine which direction goes downhill, take a step, and repeat.

5. What Is a Gradient?

A gradient tells us how the cost changes when model parameters change. For Linear Regression we consider dJ/db0 and dJ/db1. These values indicate how the parameters should move to reduce the cost.

6. Gradient Descent Update Rule

General rule:

new parameter = old parameter - learning rate * gradient

For Linear Regression:

b0 = b0 - alpha * (dJ/db0)

b1 = b1 - alpha * (dJ/db1)

Alpha is the learning rate.

7. Learning Rate

The learning rate controls the size of each parameter update.

```
Very small -> slow convergence
Very large -> may jump over the minimum or fail to converge
Suitable -> steady movement toward low cost
```

Learning rate is a hyperparameter of the optimization process.

8. Gradients for Linear Regression

For J = (1/(2n)) SUM((y_i - y_hat_i)^2), with y_hat_i = b0 + b1*x_i:

dJ/db0 = -(1/n) * SUM(y_i - y_hat_i)

dJ/db1 = -(1/n) * SUM(x_i * (y_i - y_hat_i))

The important intuition is that the gradients use prediction errors to determine how b0 and b1 should change.

9. Small Gradient Descent Example

Suppose b0 = 0 and b1 = 0. The model predicts 0 for every input. For a student who studied 5 hours and scored 65:

```
Actual = 65
Predicted = 0
Error = 65
```

The model is poor. Gradient Descent uses errors across the training data to update b0 and b1. Repeating the process produces a better-fitting line.

10. Batch Gradient Descent

Batch Gradient Descent uses the entire training dataset to calculate each update.

```
All training examples -> Calculate gradient -> One update -> Repeat
```

For very large datasets this can be expensive. Other variants include Stochastic Gradient Descent and Mini-Batch Gradient Descent.

11. Normal Equation

Gradient Descent is not the only solution. Ordinary least-squares Linear Regression has an analytical solution called the Normal Equation:

theta = (X^T X)^(-1) X^T y

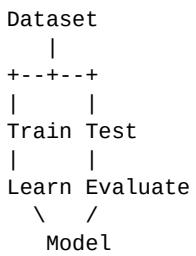
X is the feature matrix, y is the target vector, and theta contains the learned parameters. For simple Linear Regression, theta contains the intercept and slope.

12. Gradient Descent vs Normal Equation

Gradient Descent	Normal Equation
Iterative optimization	Direct mathematical solution
Requires learning rate	No learning rate
Repeated updates	No iterative updates
Useful for large feature sets	Matrix computation can become expensive with many features

13. Train/Test Split

A model should not be evaluated only on the data it was trained on. We split data so part is held out for evaluation.



A common example is 80% training and 20% testing, but the appropriate split depends on the problem.

14. Training Set

The training set is used to learn model parameters such as b0 and b1.

15. Test Set

The test set contains examples not used for fitting. It helps estimate how well the model generalizes to unseen data.

16. Evaluation Metrics

MAE - Mean Absolute Error

MAE = (1/n) * SUM(|y_i - y_hat_i|)

Measures average absolute prediction error and is easy to interpret in the original target units.

MSE - Mean Squared Error

MSE = (1/n) * SUM((y_i - y_hat_i)^2)

Squares errors, so large errors receive greater influence.

RMSE - Root Mean Squared Error

RMSE = sqrt(MSE)

Returns to the original target units while retaining MSE's stronger sensitivity to large errors.

R2 - Coefficient of Determination

R2 = 1 - [SUM((y_i - y_hat_i)^2) / SUM((y_i - y_mean)^2)]

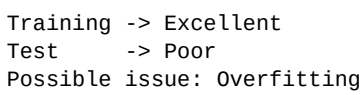
Measures how much target variation is explained by the model relative to a mean-prediction baseline. R2 is not the same as percentage prediction accuracy.

17. MAE vs MSE vs RMSE

Metric	Main idea	Large errors
MAE	Average absolute error	Linear influence
MSE	Average squared error	Strong penalty
RMSE	Square root of MSE	Strong penalty; original units

18. Overfitting

Overfitting occurs when a model performs very well on training data but poorly on unseen data. It may have learned training-specific details or noise rather than general patterns.



19. Underfitting

Underfitting occurs when a model is too simple or insufficiently trained to capture the underlying pattern.

```
Training -> Poor
Test     -> Poor
Possible issue: Underfitting
```

20. Good Generalization

```
Training -> Good
Test     -> Good
Goal -> Generalization
```

The goal of ML is not simply the lowest training error. The goal is good performance on new, unseen data.

21. Regularization - Introduction

Regularization reduces overfitting by adding a penalty for large model coefficients. It encourages simpler models.

Ridge / L2: penalizes the sum of squared coefficients.

Lasso / L1: penalizes the sum of absolute coefficient values.

A regularized objective can be viewed as: **prediction loss + regularization penalty**. Ridge and Lasso will be studied in depth later.

22. Complete Linear Regression Mental Model

```
DATA
 |
Split Dataset
 /          \
Training    Testing
 |
Linear Regression
 |
y_hat = b0 + b1*x
 |
Predictions
 |
Calculate Loss
 |
MSE
 |
Gradient Descent
 |
Update b0, b1
 |
Repeat
 |
Trained Model
 |
Evaluate on Test Data
 |
MAE / MSE / RMSE / R2
 |
Check Generalization
 |
Underfit / Good Fit / Overfit
 |
Regularization if needed
```

23. Revision Checklist

[ ] What is Machine Learning?

[ ] AI vs ML vs Deep Learning

[ ] Types of Machine Learning

[ ] Regression vs Classification

[ ] What Linear Regression does

[ ] Intercept and slope

[ ] Prediction and residual

[ ] Cost function

[ ] MSE

[ ] Gradient Descent

[ ] Gradient

[ ] Learning rate

[ ] Batch Gradient Descent

[ ] Normal Equation

[ ] Train/Test Split

[ ] MAE, MSE, RMSE, R2

[ ] Overfitting

[ ] Underfitting

[ ] Regularization

24. Final Mental Model

Do not memorize Linear Regression only as y = mx + c. The deeper idea is:

```
Data -> Model -> Prediction -> Measure Error -> Optimize Parameters -> Evaluate on Unseen Data -> Check Gener
```

This same learning pattern will reappear in Logistic Regression, Neural Networks, Deep Learning, and many other ML methods.

25. Next Learning Block

```
Linear Regression Practical Implementation
        |
From-scratch implementation
        |
scikit-learn implementation
        |
Feature scaling
        |
Polynomial Regression
        |
Regularization in depth
        |
Classification
        |
Logistic Regression
```